

E-Ticaret Platformlarında Saat Ürün Verilerinin Çıkarımı: Modern Güvenlik Duvarlarını (Anti-Bot) Aşma ve Yapılandırılmış Veri Elde Etme Stratejileri Üzerine Kapsamlı Araştırma Raporu

Kapsamlı Sorun Tanımı ve Ekosistem Analizi

Günümüz e-ticaret ekosisteminde, özellikle lüks tüketim malları, saatler ve mücevherat gibi yüksek değerli ürün segmentlerinde faaliyet gösteren platformlardan (örneğin Chrono24, Jomashop ve benzeri küresel veya yerel pazar yerlerinden) otomatik yöntemlerle veri elde edilmesi, teknolojik bir silahlanma yarışına dönüşmüştür.¹ Geçmiş yıllarda web kazıma (web scraping) işlemleri, basit bir HTTP isteği gönderilmesi, dönen HTML kodunun ayrıştırılması ve istenen verinin XPath veya CSS seçicileri aracılığıyla çekilmesi kadar basit bir operasyonken; 2026 yılı itibarıyla bu geleneksel mimariler tamamen işlevsiz hale gelmiştir.² E-ticaret platformları, rekabetçi istihbaratı engellemek, dinamik fiyatlandırma stratejilerini korumak ve sunucu kaynaklarını optimize etmek amacıyla yapay zeka destekli, çok katmanlı bot tespit sistemlerini (anti-bot) altyapılarına entegre etmişlerdir.²

Bir saat ürününün bağlantısından (link) spesifik verileri (marka, model, mekanizma tipi, güç rezervi, kasa çapı, su geçirmezlik seviyesi, kadran rengi ve anlık fiyat) otomatik olarak çekmeyi hedefleyen herhangi bir girişimin önündeki temel engel, bu sistemlerin milisaniyeler içinde gerçekleştirdiği görünmez doğrulama süreçleridir. Cloudflare, Akamai, DataDome, Kasada, PerimeterX (HUMAN Security) ve F5 BIG-IP gibi endüstri standardı güvenlik duvarları, gelen bir ziyaretçinin insan mı yoksa otonom bir yazılım mı olduğunu analiz ederken tek bir parametreye güvenmezler.³ Bu analiz, kullanıcının ağ geçmişi (IP itibarı), Aktarım Katmanı Güvenliği (TLS) şifreleme parametrelerine, tarayıcı düzeyindeki donanım parmak izlerinden (fingerprinting), fare ve klavye etkileşim dinamiklerine kadar uzanan devasa bir sinyal ağını kapsar.⁵

Bu araştırma raporu, modern e-ticaret güvenlik duvarlarını aşmak için ağ, protokol ve tarayıcı katmanlarında uygulanabilecek en güncel siber kaçınma (evasion) tekniklerini derinlemesine analiz etmektedir. Güvenlik katmanlarının anatomisi deşifre edildikten sonra, aşılmış bir sistemden saat özelliklerini son derece istikrarlı ve tasarım değişikliklerinden etkilenmeyecek şekilde elde edebilmek için Schema.org (JSON-LD) tabanlı yapılandırılmış veri çıkarımı ve Büyük Dil Modelleri (LLM) ile Doğal Dil İşleme (NLP) destekli ayrıştırma stratejileri incelenecektir.⁸ Raporda sunulan mimari örüntüler, yüksek ölçekli, kesintisiz ve tespit edilemeyen veri çıkarım boru hatları (data pipelines) tasarlamak için gereken teorik ve pratik temelleri sağlamaktadır.

Modern Anti-Bot Sistemlerinin Çok Katmanlı Anatomisi

E-ticaret platformlarını koruyan modern anti-bot sistemleri, meşru kullanıcılara görünmez olacak şekilde tasarlanmış olsalar da, hiçbir teknoloji arkasında iz bırakmadan çalışmaz. Her bir bot koruma sağlayıcısı, tarayıcınızda veya ağ trafiğinizde okumasını bilen bir göz için ürün etiketi kadar belirgin olan parmak izleri bırakır.⁴ Bu izler; spesifik HTTP yanıt başlıkları (response headers), tarayıcıya enjekte edilen benzersiz çerezler (cookies), üçüncü taraf betik URL'leri ve window nesnesine bağlanan global JavaScript değişkenleri şeklinde ortaya çıkar.⁴ Örneğin, Cloudflare web üzerindeki en yaygın koruma katmanı olarak en bilindik izleri bırakırken; kurumsal finans ve e-ticaret sitelerinde uzmanlaşan Akamai, alt-milisaniye hızında makine öğrenimi tabanlı davranışsal analiz yapan DataDome veya tersine mühendisliği zorlaştıran Kasada, çok daha karmaşık tespit vektörleri kullanır.³ Bir saat e-ticaret sayfasına yapılan otomatize bir isteğin geçmesi gereken dört temel savunma katmanı bulunmaktadır.

Ağ Katmanı ve IP İtibarı Analizi (IP Reputation)

Bir HTTP veya HTTPS isteğinin hedef sunucuya ulaşmadan önce karşılaştığı ilk mutlak savunma hattı ağ itibarı analizidir. Modern güvenlik sistemleri, is-antibot gibi tespit mekanizmalarında da görüldüğü üzere, gelen isteğin kaynak IP adresini devasa küresel veritabanlarıyla anlık olarak çapraz sorguya tabi tutar.⁵ Eğer gelen istek Amazon Web Services (AWS), DigitalOcean, Google Cloud, Linode veya benzeri bilinen bir bulut barındırma hizmetinden (veri merkezi / datacenter) geliyorsa, sistem bu IP adresini varsayılan olarak "insan dışı" ve yüksek riskli olarak işaretler.⁵ Veri merkezi IP'lerinden gelen trafik, e-ticaret sitesinin konfigürasyonuna bağlı olarak ya doğrudan 403 Forbidden (Yasaklandı) veya 429 Too Many Requests (Çok Fazla İstek) hata kodlarıyla reddedilir ya da kullanıcıya aşılması imkansız yakın, sürekli yenilenen bir CAPTCHA döngüsü sunulur.⁵ Konut tipi (Residential) trafik ise gerçek bir ev interneti abonesinden geldiği için yapısal olarak çok farklı bir ağ davranışı sergiler ve sistem tarafından başlangıçta daha yüksek bir güven skoruyla (trust score) karşılanır.⁵ Ağ katmanı bariyeri doğru bir proxy mimarisiyle aşılmadığı sürece, uygulanan hiçbir ileri düzey yazılım hilesi işe yaramayacaktır.

Protokol Katmanı Savunmaları: TLS Parmak İzi (JA3 ve JA4 Algoritmaları)

Ağ katmanı geçildiğinde, sistem iletişimin nasıl şifrelendiğini incelemeye başlar. Güvenli web iletişimi sağlayan Aktarım Katmanı Güvenliği (TLS) protokolü, HTTPS bağlantısının temelini oluşturur. Bir istemci (bu bir Python betiği, bir cURL komutu veya Chrome tarayıcısı olabilir) sunucuya bağlandığında, el sıkışma (handshake) sürecini başlatmak için açık metin (cleartext) formatında bir ClientHello mesajı gönderir.¹¹ Bu ClientHello mesajının içerdiği yapı taşları; desteklenen TLS sürümü, şifre paketleri (cipher suites), desteklenen eliptik eğriler (elliptic curves) ve ALPN, SNI gibi çeşitli protokol uzantılarıdır.¹¹

Farklı kütüphaneler (örneğin Node.js'in tls modülü, Go dilinin crypto/tls modülü veya Python'un

yerleşik HTTP modülleri) bu ClientHello paketini kendi iç standartlarına göre tamamen farklı sıralamalar ve kombinasyonlarla oluştururlar.¹¹ Endüstri standardı haline gelen JA3 TLS parmak izi algoritması, bu beş temel alanı alır (TLS Versiyonu, Şifre Paketleri, Uzantılar, Eliptik Eğriler ve EC Nokta Formatları), bunları ondalık değerlerden oluşan uzun bir dizeye dönüştürür ve ardından bu dizeyi MD5 hash algoritmasıyla şifreleyerek 32 karakterlik benzersiz bir parmak izi üretir.¹¹ E-ticaret platformunun arkasındaki WAF (Web Application Firewall), bu 32 karakterlik hash değerini veritabanındaki bilinen tarayıcı profilleriyle karşılaştırır. Python requests kütüphanesi kullanılarak bir istek yapıldığında ve User-Agent başlığı "Mozilla/5.0 Chrome..." olarak ayarlandığında, WAF sistemi Chrome'a ait User-Agent ile OpenSSL'e ait JA3 parmak izi arasındaki asimetriyi saniyesinde tespit ederek bağlantıyı sonlandırır.¹¹

Ancak siber güvenlik sistemleri JA3 ile yetinmemiştir. JA3'ün zayıf noktalarını (örneğin modern tarayıcıların uzantı sıralamasını rastgele değiştirerek -greasing- basit string eşleşmelerini bozması) gidermek için daha yeni ve sağlam bir algoritma olan JA4 geliştirilmiştir.¹¹ JA4 algoritması, şifre paketleri ve uzantılar gibi alanları kanonikleştirmekle (normalize etmekle) kalmaz, aynı zamanda TLS el sıkışmasını TCP katmanı özellikleri, Sunucu Adı Belirtme (SNI) davranışları ve HTTP/2 ayarlarıyla entegre ederek, istemcinin çok daha kapsamlı ve 36 karakterlik, yüksek hassasiyetli bir kimliğini çıkarır.¹¹ Yüksek profilli hedef e-ticaret siteleri, yapay zeka kazıyıcılarına karşı JA4 parmak izini kullanarak, sadece User-Agent değiştirmenin veya basit başlıklı istekler atmanın ötesine geçen güçlü bir duvar örmüşlerdir.¹⁵ Bu bağlamda, JA4 parmak izinin taklit edilebilmesi için ağır tarayıcı altyapılarına, gelişmiş proxy mimarilerine veya özel olarak modifiye edilmiş HTTP istemcilerine mutlak ihtiyaç duyulmaktadır.¹⁵

Tarayıcı Katmanı ve Donanım Parmak İzi (JavaScript Fingerprinting)

Protokol katmanında herhangi bir anormallik tespit edilmezse, sunucu istemciye asıl sayfa içeriğini göndermeden önce veya sayfayla entegre olarak çalışan karmaşık JavaScript doğrulama betikleri (challenges) gönderir.⁵ Bu betikler, bağlanan cihazın sanal bir ortam mı (Docker container, başsız sunucu) yoksa fiziksel bir bilgisayar mı olduğunu anlamak için donanım özelliklerini piksel ve bayt düzeyinde sorgular. Scrapfly platformunun detaylı analizlerine göre, anti-bot sistemleri 30.000'den fazla kohezif tarayıcı sinyalinin kontrol etmektedir.⁷ Tarayıcı katmanındaki temel tespit vektörleri şu şekilde kategorize edilir:

1. **Otomasyon Bayraklarının İfşası (navigator.webdriver):** Selenium, Playwright veya Puppeteer gibi otomasyon çerçeveleri varsayılan olarak başlatıldıklarında, tarayıcının JavaScript motorundaki navigator.webdriver özelliğini true olarak ayarlarlar.⁷ Kurumsal güvenlik sistemleri, bu tek bir bayrağın varlığını kontrol ederek otomasyonu anında ve kesin olarak ispatlar.
2. **Kullanıcı Ajanı (User-Agent) ve İstemci İpuçları (Client Hints) Çatışması:** Başsız (Headless) modda çalışan Chromium tabanlı tarayıcılar, kimlik dizelerine HeadlessChrome ibaresini eklerler.¹⁴ Basit hilelerle User-Agent değiştirilse bile, Google'ın devreye soktuğu User-Agent Client Hints teknolojisi, işletim sistemi mimarisini ve tarayıcı sürümünü daha alt seviyeden ifşa ederek, beyan edilen başlıklarla gerçek donanım arasındaki

uyumsuzlukları ortaya çıkarır.⁷

3. **Medya, Grafik ve Donanım Çelişkileri:** Gerçek bir kullanıcının masaüstü tarayıcısında her zaman PDF Görüntüleyici gibi varsayılan eklentiler (plugins) bulunur.⁷ Ancak otomatize edilmiş başsız tarayıcılarda navigator.plugins dizisi genellikle boştur.⁷ Daha derinlemesine incelemelerde, HTML5 Canvas API'si ve WebGL kullanılarak ekran kartı (GPU) modelinin ve render yeteneklerinin parmak izi çıkarılır. WebGL sağlayıcı dizeleri veya donanım seviyesindeki ses (AudioContext) özellikleri, sanal bir makine ile gerçek bir grafik kartı arasındaki farkı matematiksel olarak kanıtlar.⁵

Davranışsal Analiz ve Sezgisel Katman (Behavioral Heuristics)

Makine öğrenimi ve yapay zeka modelleriyle desteklenen sistemler (DataDome, Kasada, PerimeterX), sadece statik verileri değil, kullanıcının dinamik davranış kalıplarını da sürekli izler. Bir e-ticaret sitesinde saat özelliklerini çekmek isteyen bir bot, sayfaya bağlandığı an anında sayfa sonuna kaydırma yapar, ürün özelliklerini içeren HTML elementlerini (DOM üzerinden) milisaniyeler içinde kopyalar ve bağlantıyı kapatır. Gerçek bir insanın doğal okuma hızı, farenin ivmelenme eğrileri (mouse trajectory), tıklama ritimleri ve sayfalar arası gezinme süresi (dwell time) botların deterministik ve mekanik hızından tamamen farklıdır.⁵ Fare hareketlerinin yokluğu veya sayfanın doğal olmayan bir hızda işlenmesi, donanım parmak izi kusursuz olsa bile sistemin 403 Forbidden engellemesi yapmasına yol açan en belirgin anormalliklerden biridir.⁷

Güvenlik Duvarlarını Aşmak İçin Çözüm Mimarileri ve Araçlar

E-ticaret platformlarındaki bu devasa savunma katmanları analiz edildiğinde, eski nesil web kazıma yöntemlerinin veya temel Python betiklerinin saat ürünü detaylarını elde etmede neden yetersiz kaldığı net biçimde anlaşılmaktadır. Bu sistemleri bypass etmek için, müdahalenin derinliğine bağlı olarak yapılandırılmış modern araçlar kullanılmalıdır. Geleneksel araçların ve basit eklentilerin sınırlarını anlamak, yeni nesil sürücüsüz (driverless) ve protokol manipülasyonu yapan mimarilerin neden vazgeçilmez olduğunu ortaya koyar.

Geleneksel Otomasyonun ve Stealth Eklentilerinin Yapısal Çöküşü

Geçmiş yıllarda web kazıyıcılar, Playwright veya Selenium tabanlı araçların tespit edilmesini engellemek için puppeteer-extra-plugin-stealth (Node.js için) veya playwright-stealth (Python için) gibi üçüncü taraf eklentilere güvenmekteydi.⁷ Ancak 2026 koşullarında, bu stealth (gizlilik) eklentileri son derece kısıtlı bir koruma sağlamaktadır.

Bu eklentilerin çalışma mantığı, sayfa yüklenmeden hemen önce tarayıcıya çeşitli JavaScript yamaları (shims) enjekte etmektir.¹⁷ navigator.webdriver bayrağını false olarak değiştirmek, eklenti dizilerini sahte isimlerle doldurmak ve WebGL parmak izi sonuçlarını maskelemek gibi görevleri üstlenirler.⁷ Ancak bu yöntemlerin yapısal üç büyük zafiyeti bulunmaktadır:

1. **Yalnızca Uygulama Katmanı Sınırlaması:** Stealth eklentileri sadece JavaScript (tarayıcı)

katmanında çalışır. Bot tespit sistemlerinin ağ (IP itibarı) ve protokol (TLS JA3/JA4) katmanlarındaki analizlerine hiçbir şekilde müdahale edemezler.⁷

2. **Derin Chrome API'lerinin Eksikliği:** Başsız (Headless) Chromium, gerçek bir masaüstü Chrome tarayıcısında bulunan chrome.app, chrome.csi veya chrome.loadTimes gibi özel yerleşik API'lerden yoksundur.⁷ Anti-bot sistemleri, JavaScript yamalarını atlayarak doğrudan bu derin C++ tabanlı API'lerin varlığını sorgular ve yoklukları durumunda sistemin bir bot olduğunu kesinleştirir.⁷
3. **Milisaneye Gecikmesi Tespitleri:** Güvenlik sistemleri, stealth modlarının kullandığı JavaScript yamalarını uygulandıkları an, 50 milisaniyenin altında bir sürede tespit edebilecek kapasiteye ulaşmıştır.³ Yamalanmış bir navigator objesi ile doğal bir objenin bellekte kapladığı alan ve string dönüşüm özellikleri (örneğin fonksiyonların [native code] dönmesi gerekirken müdahale edilmiş kod dönmesi) sistemler tarafından kolayca teşhis edilir.³

Yeni Nesil Sürücüsüz (Driverless) ve CDP Tabanlı Mimari: Nodriver ve Zendriver

WebDriver protokolünün tespit edilebilir doğasını kökünden çözmek için, tarayıcılarla iletişim kurma paradigması tamamen değişmiş ve Chrome Geliştirici Araçları Protokolü (CDP - Chrome DevTools Protocol) tabanlı "sürücüsüz" (driverless) mimariler öne çıkmıştır.¹⁸

Bu ekosistemin yaratıcısı olan **Nodriver**, efsanevi undetected-chromedriver projesinin resmi halefi olarak geliştirilen, Python tabanlı ve asenkron öncelikli bir kütüphanedir.¹⁸ Nodriver, geleneksel bir WebDriver çalıştırılabilir dosyasına (executable) ihtiyaç duymadan, doğrudan işletim sistemi seviyesindeki tarayıcı işlemiyle (browser process) CDP üzerinden websocket veya pipe aracılığıyla konuşur.¹⁸ Bu yaklaşım, anti-bot sistemlerinin Selenium veya Playwright'in bilindik komut zincirlerini tespit etmesini engeller. Ancak Nodriver projesindeki güncelleme yavaşlığı, topluluğun **Zendriver** adında daha gelişmiş ve sürekli güncellenen bir çatall (fork) oluşturmasına zemin hazırlamıştır.²⁰

Bu araçların gerçek dünya senaryolarında, herhangi bir özel eklenti veya proxy kullanılmadan ("out-of-the-box") elde ettiği performansları karşılaştırmak, modern güvenlik duvarlarının gücünü anlamak açısından kritiktir. Çeşitli anti-bot sistemleriyle korunan genel e-ticaret ve kurumsal hedefler üzerinde yapılan standart testlerin sonuçları aşağıdaki tabloda özetlenmiştir¹⁸.

Otomasyon Aracı	Cloudflare Başarısı	Datadome Başarısı	Akamai Başarısı	Cloudfront Başarısı	Genel Başarı Oranı

Selenium (Standart)	Başarısız (✗)	Başarısız (✗)	Başarılı (✓)	Başarısız (✗)	%25
Playwright (Standart)	Başarısız (✗)	Başarısız (✗)	Başarısız (✗)	Başarılı (✓)	%25
Nodriver	Başarısız (✗)	Başarısız (✗)	Başarısız (✗)	Başarılı (✓)	%25
Zendriver	Başarılı (✓)	Başarısız (✗)	Başarılı (✓)	Başarılı (✓)	%75

Tablodan da açıkça görülebileceği üzere, Zendriver şu an açık kaynaklı otomasyon araçları arasında en yüksek tespit edilememe yeteneğine sahiptir. Cloudflare ve Akamai gibi devasa sistemleri varsayılan ayarlarıyla aşabilmektedir.¹⁸ Ancak en kritik bulgu, **Datadome** sisteminin gücüdür. Test edilen tüm araçlar Datadome karşısında %0 başarı göstermiş ve anında bloke edilmişlerdir.¹⁸ Jomashop gibi yüksek profilli saat e-ticaret siteleri DataDome veya PerimeterX gibi üst düzey korumalar kullanabileceğinden, sorunu çözmek için yalnızca Zendriver kullanmak yetersiz kalacak, ağ katmanı manipülasyonlarıyla desteklenmesi gerekecektir.

Protokol Katmanı Evasion: curl_cffi ve tls-client Yüksek Hızlı İstemcileri

Bir saat e-ticaret platformunda, eğer ürün özellikleri doğrudan HTML ile sunulmuyor, bunun yerine mobil uygulamayı veya dinamik ön yüzü destekleyen bir JSON API uç noktası (endpoint) üzerinden geliyorsa; ağır, kaynak tüketen ve yavaş olan tarayıcı otomasyonlarını (Zendriver vb.) çalıştırmak verimsizdir. Veri madenciliğinde ideal olan, doğrudan HTTP istekleri atmaktır. Ancak daha önce açıklandığı gibi, TLS ve HTTP/2 parmak izleri nedeniyle standart Python requests veya httpx kütüphaneleri anında tespit edilir.

Bu zafiyeti gidermek için iki devrimsel kütüphane geliştirilmiştir:

1. **tls-client:** Go dili tabanlı özel bir HTTP istemcisinin Python bağlayıcısıdır. Çeşitli modern tarayıcıların TLS parmak izlerini kopyalayarak, anti-bot sistemlerinin ClientHello mesajlarını analiz ettiğinde gerçek bir Chrome veya Safari ile karşılaştığını zannetmesini sağlar.²²
2. **curl_cffi:** curl-impersonate projesinin gücünü CFFI (C Foreign Function Interface) üzerinden Python'a taşıyan son derece güçlü bir mimaridir.²⁵ cURL'ün standart TLS kütüphanelerini modifiye ederek, uzantıları, eliptik eğrileri ve şifre paketlerini (cipher suites) gerçek tarayıcıların yapılandırmasıyla birebir aynı olacak şekilde ayarlar. Üstelik

HTTP/2 standartlarını, çoklama (multiplexing) mekanizmalarını ve sözde başlıkları (pseudo-headers) tarayıcı mantığına uyarlar.²³

Karşılaştırmalı Analiz: Performans ve yetenek açısından curl_cffi, tls-client kütüphanesine kıyasla önemli üstünlükler sunar. Asenkron asyncio desteği, bağlantı havuzlama (connection pooling), otomatik dekompresyon ve SOCKS5 proxy desteği ile yüksek ölçekli veri çekimi için idealdir.²⁶ Üstelik curl_cffi, TLS PSK (Pre-Shared Key) uzantısı gibi spesifik konulardaki bazı zafiyetleri dışında, API çağrıları için Cloudflare veya Akamai'nin JA3/JA4 algoritmalarını sessizce ve yüksek bir hızla geçer.²⁵

Yönetilen Bulut Tarayıcı Mimarileri (Cloud Browser APIs)

Eğer hedef saat sitesi Datadome gibi ultra-agresif bir koruma kullanıyor, gelişmiş davranışsal analizler (heuristics) yapıyor ve sayfa yüklemesinde karmaşık CAPTCHA sistemlerini (Slider veya Turnstile) zorunlu tutuyorsa, açık kaynaklı kodların sınırına ulaşılır. Bu noktada profesyonel mimariler, Yönetilen Bulut Tarayıcı API'lerine (Scrapfly, Browserless, Scrape.do) yönelirler.¹

- **Scrapfly ve "Scrapium" Teknolojisi:** Standart başsız tarayıcıların tespit edilebilir sorununu JavaScript yamalarıyla çözmek yerine, tarayıcının çekirdeğine inerler. Scrapfly, C++ ve işletim sistemi (OS) seviyesinde 550'den fazla özel kaynak kod yaması barındıran "Scrapium" adlı özel bir Chromium çatallı geliştirmiştir.³ Bu teknoloji, Canvas, WebGL, Audio Context ve ekran parametreleri dahil olmak üzere 30.000'den fazla tarayıcı sinyalinin yapısal bir bütünlük (coherent signal spoofing) içinde taklit eder.⁷
- **Browserless ve BrowserQL:** Bir diğer ileri düzey platform olan Browserless, ağ seviyesindeki TLS parmak izini tarayıcı kimliğiyle kusursuz bir şekilde eşleştirir. "BrowserQL" adı verilen gizli bir hata ayıklama (debugger) protokolü kullanarak otomasyon araçlarının bıraktığı izleri siler.¹¹ Üstelik, içine gömülü iframeler (shadow DOM) içindeki CAPTCHA'ları dahi üçüncü taraf hizmetlere gerek duymadan otomatik çözen entegre bir yapay zeka modülü barındırır.¹¹

Bu servisler, mevcut bir Python betiğine connect_over_cdp() veya WebSocket üzerinden basit bir entegrasyonla dahil edilir; oturumları (sessions) ve çerezleri (cookies) otomatik kaydederek IP rotasyonlarından kaynaklı ani değişiklikleri gizler.⁶

Ağ Stratejisi: Proxy Havuzları ve IP Rotasyon Mimarisi

Bir saat ürünü sayfasından veri çeken sistemin sürekliliği, kullanılan IP mimarisinin kalitesine doğrudan bağlıdır. Hedef platformun güvenlik duvarı aşıldığında bile, aynı IP adresinden dakikalar içinde yüzlerce istek yapıldığında sistem davranışsal hız sınırlarını (rate limits) aşacak ve IP adresini kara listeye alacaktır.²⁸ Bu nedenle yüksek hacimli veri madenciliği, dönen proxy havuzları (rotating proxy pools) kullanılarak isteklerin çok sayıda farklı bağlantı noktasına dağıtılmasını gerektirir.¹

Farklı proxy türleri ve bunların e-ticaret sitelerindeki tespit edilebilirlik matrisi aşağıda

sunulmuştur:

Proxy Türü	Kaynak Yapısı	Güven (Trust) Skoru	Maliyet	Tespit Edilebilirlik	Kullanım Senaryosu
Veri Merkezi (Datacenter)	AWS, DigitalOcean vb. bulut sunucuları.	Düşük	Düşük	Çok Yüksek (ASN seviyesinde kolayca engellenir)	Güvenlik duvarı olmayan statik siteler, temel veri çekimi.
Konut Tipi (Residential)	İSS (İnternet Servis Sağlayıcısı) tarafından gerçek evlere atanan IP'ler.	Yüksek	Yüksek	Düşük	Kurumsal e-ticaret platformları, zorlu WAF'ları aşmak, bölgesel kısıtlamaları geçmek.
Mobil (4G/5G)	GSM operatörlerini n cep telefonlarına atadığı hücresel IP'ler.	Çok Yüksek	Çok Yüksek	Çok Düşük (CGNAT mekanizması nedeniyle)	Datadome ve Kasada gibi en üst düzey korumaları ve CAPTCHA döngülerini aşmak.

Mobil Proxy'lerin CGNAT Kalkanı Etkisi

E-ticaret veri çekimi için en üst düzey çözüm Mobil Proxy'lerdir.¹⁰ Mobil cihazların kullandığı IP adreslerinin son derece yüksek bir güven skoruna sahip olmasının ardında yapısal bir ağ protokolü yatar: Taşıyıcı Seviyesinde Ağ Adresi Çevirisi (CGNAT). IPv4 adreslerinin küresel çapta tükenmiş olması nedeniyle, telekomünikasyon şirketleri tek bir mobil IP adresini aynı anda binlerce gerçek kullanıcıya tahsis eder.¹⁰

Cloudflare veya Datadome gibi sistemler, bir mobil IP'den gelen şüpheli, yüksek hacimli veya

otomatize edilmiş bir trafik tespit ettiklerinde, normalde yapacakları gibi o IP adresini tamamen ve kalıcı olarak engelleyemezler. Çünkü o mobil IP'nin engellenmesi, o an aynı IP'yi paylaşan binlerce meşru ve potansiyel müşterinin de (gerçek insanlar) e-ticaret sitesine erişiminin kesilmesi anlamına gelir.¹⁰ Bu asimetrik risk, e-ticaret sitelerinin algoritmalarını mobil IP aralıklarına karşı çok daha toleranslı (lenient) çalışmaya mecbur bırakır. Dolayısıyla, mobil veya konut tipi proxy rotasyonu kullanılarak gerçekleştirilen istekler, ağ katmanındaki tüm itibar engellerini zahmetsizce aşar.¹⁰

Sistem kararlılığını sağlamak adına, proxy havuzlarını yönetirken Dağıtık Sistem Tasarım Desenlerinden (Microservices Design Patterns) biri olan Devre Kesici (Circuit Breaker) mantığı uygulanmalıdır. Hata veren veya geçici olarak CAPTCHA'ya düşen bir proxy IP'si sistemden hızla izole edilmeli ve istekler temiz IP'ler üzerinden yeniden yönlendirilmelidir.²⁹ Ayrıca, bağlantı yapılan proxy IP'sinin coğrafi konumu ile tarayıcının başlıklarında gönderilen saat dilimi (timezone) ve dil özelliklerinin tam bir eşleşme içinde olması, konum uyumsuzluğu (geo-mismatch) alarmlarını susturmak için kritik bir detaydır.⁷

Saat Özelliklerinin Yapılandırılmış Çıkarım (Parsing) Stratejileri

Ağ (IP), protokol (JA4) ve tarayıcı (CDP/Scrapium) düzeyindeki tüm güvenlik katmanları aşıldıktan sonra, e-ticaret sayfasının kaynak kodu (HTML DOM) elde edilir. Sorunun odak noktası olan "saat özelliklerini veri olarak çekmek" bu aşamada başlar.

Geleneksel web kazıma metodolojisinde geliştiriciler, HTML içindeki sınıflara (class) dayalı karmaşık XPath veya CSS seçicileri yazarlar (örneğin: `div.product-card > span.price-wrapper > span.value`).³¹ Ancak modern geliştirme süreçlerinde bu yaklaşım son derece kırılgan ve sürdürülemezdir. E-ticaret siteleri, dönüşüm oranlarını artırmak için sürekli A/B testleri uyguluyorlar ve Tailwind CSS gibi araçlarla sınıfları sürekli olarak değiştirirler (örneğin sınıf adının aniden `text-sm font-bold` olarak güncellenmesi).³¹ Bu durum, sayfa mimarisinde "layout churn" (sürekli düzen değişimi) adı verilen olguya neden olur ve yazılan veri çekme kodlarının anında hata verip çökmesiyle sonuçlanır.³¹

Bu kırılganlığı önlemek ve bir saat ürününün spesifikasyonlarını hatasız, istikrarlı ve düzen değişikliklerinden bağımsız olarak elde etmek için, e-ticaret endüstrisinin kendi yarattığı arama motoru optimizasyonu (SEO) zorunlulukları kendi aleyhine kullanılmalıdır: Schema.org (JSON-LD) Çıkarımı.

1. Kesin Çözüm Mimarisi: Schema.org ve JSON-LD Metadata Analizi

Küresel çapta 10 milyondan fazla web sitesinin kullandığı Schema.org, internetteki içeriklerin arama motorları (Google, Bing) tarafından doğru anlaşılmasını sağlayan standartlaştırılmış, makine tarafından okunabilir bir veri sözlüğüdür.⁹ Lüks tüketim veya genel perakende fark etmeksizin, Chrono24 ve Jomashop dahil tüm büyük e-ticaret platformları, ürünlerinin Google Alışveriş (Google Merchant Center) sekmesinde görünmesi, fiyat ve stok durumlarının arama

sonuçlarında "zengin snippet" (rich result) olarak çıkması için ürün sayfalarına yapılandırılmış veri gömmek zorundadır.³³

Bu yapılandırılmış veri, e-ticaret sayfalarının HTML kaynak kodunun içine, genellikle <head> etiketleri arasında <script type="application/ld+json"> formatında entegre edilir.⁹ E-ticaret sitelerinde hedeflenen başlıca şema türleri şunlardır:

- **Product (Ürün) ve Merchant Listings:** Ürünün markası, stok saklama birimi (SKU), varyantları, incelemeleri ve genel tanımını içerir.³⁴
- **Offer (Teklif) ve Product Availability:** Ürünün anlık fiyatı, para birimi ve stok durumunu (InStock, OutOfStock) kesin olarak beyan eder.³⁴
- **BreadcrumbList:** Ürünün sitedeki kategorik hiyerarşisini (Örneğin: Anasayfa > Saatler > Rolex > Submariner) belirtir.³³
- **VideoObject:** Ürün sayfasında bir inceleme videosu varsa süresi, kapak fotoğrafı ve içeriği hakkında metadata sağlar.³⁴

Saat Ürününden JSON-LD Örneği:

Bir lüks saat sayfasından veri çekildiğinde, karmaşık HTML ağacıyla uğraşmak yerine, arka planda şu şekilde bir JSON nesnesi elde edilir ⁹:

JSON

```
<script type="application/ld+json">
{
  "@context": "https://schema.org/",
  "@type": "Product",
  "name": "Rolex Submariner Date 41mm",
  "image": [
    "https://example.com/saat/rolex-sub.jpg"
  ],
  "description": "Oyster, 41 mm, Oystersteel. Reference 126610LN. Su geçirmezlik 300m.",
  "sku": "126610LN",
  "brand": {
    "@type": "Brand",
    "name": "Rolex"
  },
  "offers": {
    "@type": "Offer",
    "url": "https://example.com/rolex-submariner",
    "priceCurrency": "USD",
```

```
"price": "10250.00",  
"availability": "https://schema.org/InStock"  
}  
}  
</script>
```

Bu Yaklaşımın Stratejik Avantajı: Veri çekme kodu, yalnızca sayfa içindeki script etiketini bulmaya ve içindeki metni yerleşik bir JSON ayrıştırıcısı ile işlemeye indirgenir. Sayfa tasarımı tamamen yenilense, sınıflar silinse veya menü çubukları değiştirilse dahi; e-ticaret siteleri Google sıralamalarını (SEO) kaybetmemek için bu JSON-LD kodunu sayfada eksiksiz ve belirli bir standartta tutmak zorundadır.⁸ Bu durum, web kazıma süreçleri için olağanüstü derecede istikrarlı ve bozulmaz (future-proof) bir mimari sunar.

2. İleri Düzey Çözüm: Büyük Dil Modelleri (LLM) ile Eksik Veri Tamamlama

JSON-LD yöntemi fiyat, marka, ürün adı ve stok durumunu kesin olarak sağlasa da, lüks saatlerin spesifik donanımsal özellikleri (örneğin; mekanizma kalibre numarası, güç rezervi saati, toka türü, kasa materyali, cam türü, bezel tasarımı) her zaman şema işaretlemelerine dahil edilmeyebilir.⁸ Sayfa içinde karmaşık tablolar veya düzensiz paragraflar halinde dağınık halde bulunan bu detaylı özellikleri çekmek için, geleneksel kurallı (rule-based) algoritmalar yetersiz kalır. Bu engeli aşmak için Çok Modlu Büyük Dil Modelleri (MLLM) ve Yapay Zeka destekli ayrıştırma kullanılmalıdır.⁴⁰

LLM'ler, ekran görüntülerini analiz ederek (görsel algı) veya karmaşık HTML yığınları üzerinden bir saatin teknik spesifikasyonlarını eksiksiz anlama ve standardize edilmiş bir JSON çıktısına dönüştürme yeteneğine sahiptir.⁸ Ancak, e-ticaret kazımasında LLM kullanımının önünde devasa engeller vardır: Bir ürün listeleme sayfası (PLP), tüm HTML, stil dosyaları ve betiklerle birlikte 100.000'den fazla belirteç (token) tüketebilir. API maliyetlerinin yüksekliği, işlem hızının yavaşlığı, "halüsinasyon" (olmayan bir ürün verisini uydurma) riskleri bu yöntemin naif bir şekilde (tüm sayfayı LLM'e yollayarak) kullanımını imkansız kılar.⁸

Token Optimizasyonu ve LLM Boru Hattı (Pipeline) Tasarımı

Maliyetleri yüzlerce dolardan birkaç cent seviyesine indirmek ve hızı artırmak için, kapsamlı bir 4 aşamalı veri optimizasyon boru hattı kurulmalıdır⁸:

1. **Agrege Edilmiş Bağlantı Filtreleme:** Sistem, sitenin tamamını LLM'e göndermek yerine, öncelikle siteden toplanan URL dizinlerini basit bir metin listesi halinde modele verir. Modelden yalnızca potansiyel "Ürün Listeleme Sayfaları" (PLP) olan linkleri filtrelemesi istenir. Böylece işlem hacmi dramatik biçimde düşürülür.⁸
2. **Yapısal HTML Temizliği (Structural HTML Cleaning):** Seçilen ürün sayfasının HTML kodu modele gönderilmeden önce, Python'daki BeautifulSoup gibi kütüphaneler kullanılarak kod "budanır". Tüm <script>, <style>, <meta>, <link>, <nav> ve <footer> etiketleri ile HTML yorumları silinir. Sadece class, id gibi öz nitelikler bırakılır. Bu basit adım, sayfanın veri

yükünü (token miktarını) %80 oranında azaltarak 20.000 belirteç bandına düşürür.⁸

3. **NLP Destekli Akıllı Filtreleme:** Temizlenmiş HTML blokları, hafif ve ucuz bir Doğal Dil İşleme (NLP) modeli (örneğin spaCy) kullanılarak taranır ve Varlık İsmi Tanıma (NER - Named Entity Recognition) algoritmasından geçirilir. Sistem, "Para Birimi" (MONEY) ve spesifik "Ürün" (PRODUCT) varlıklarını bulduğunda, ilgili HTML düğümüne yüksek puan verir. Düzenli ifadeler (Regex) kullanılarak pazarlama metinleri ve menüler ayıklanır. İç içe geçmiş ağaç yapıları (The Tree Problem) tekilleştirilerek aynı saatin iki kere çekilmesi engellenir. En nihayetinde, sadece saatin spesifikasyon tablolarının bulunduğu yoğunlaştırılmış HTML metni (yaklaşık 5.000 token) elde edilir.⁸
4. **Şema Tabanlı Toplu Çıkarım (CSS Selector Schema Generation):** En kritik optimizasyon adımı budur. LLM modeli, veri çekmek için sürekli çağrılmaz.³² Sistem, yoğunlaştırılmış ve temizlenmiş HTML kodunu LLM'e vererek, "bu saatin özelliklerinin sayfanın neresinde durduğunu analiz etmesini ve bir CSS Seçici Şeması yazmasını" emreder. Model, saatin kalibre numarasının `div.spec-table > ul > li.caliber` altında olduğunu tespit eder ve bir JSON formatında bu seçici mantığını geri döndürür. Bu şema önbelleğe alınır. Sonraki on binlerce saat sayfası için yapay zeka tamamen devreden çıkarılır ve standart Python kütüphaneleriyle milisaniyeler içinde devasa bir veri çıkarımı yapılır.⁸

Bütünleşik Entegrasyon ve Sonuç Çıkarımları

Web kazıma, e-ticaret sitelerindeki saat ürünlerinden fiyat, referans kodu ve mekanik özellikleri çıkarma arzusunun çok ötesine geçerek; ağ mühendisliği, kriptografi (TLS/JA4), davranışsal biyometri ve yapay zeka filtreleme tekniklerinin iç içe geçtiği kompleks bir veri mühendisliği problemine dönüşmüştür.

Elde edilen veriler açıkça göstermektedir ki; güvenlik duvarları karşısında standart Selenium veya Headless Chrome kullanmak yalnızca zaman ve kaynak kaybıdır. İleri düzey korumalar (Cloudflare Turnstile, DataDome), otomasyonun yapısal eksikliklerini anında tespit etmektedir. Kullanıcının hedefine ulaşması için tasarlanması gereken veri çekim mimarisi, sorunları her katmanda tecrit etmelidir.

Ağ katmanında, CGNAT avantajından ötürü bloke edilmesi neredeyse imkânsız olan mobil proxy havuzları kullanılmalı ve devre kesici (circuit breaker) algoritmalarıyla rotasyon sağlanmalıdır. Protokol katmanında, sayfadaki JavaScript doğrulama testlerini baypas etmek veya API'leri sorgulamak için Chrome/Safari kimliğine bürünebilen `curl_cffi` HTTP istemcisi kullanılmalıdır. Daha agresif davranışsal testlerin olduğu hedeflerde ise `navigator.webdriver` sorununu kökünden kazıyan CDP tabanlı "Zendriver" veya tarayıcı düzeyinde yamalanmış (Scrapium) Yönetilen Bulut Tarayıcıları entegre edilmelidir.

Güvenlik duvarları aşıldıktan sonraki ayrıştırma aşamasında, her türlü site tasarımı değişimine karşı bağımsızlık kazanmak amacıyla arama motoru optimizasyonlarının sunduğu açık kapıdan yararlanılmalı ve temel veriler (fiyat, stok, varyant) `<script type="application/ld+json">` düğümlerindeki Schema.org standartlarından çekilmelidir. Saat kasası materyali, cam özellikleri gibi karmaşık ve şema dışı verilerin elde edilmesi ise; token limitlerini ve maliyetleri düşürmek

için URL kümeleme, HTML temizliği, hafif NLP algoritmaları ve LLM tabanlı CSS Seçici Şeması çıkarma işlemlerini barındıran akıllı bir boru hattına devredilmelidir. Bu bütünlük ve hiyerarşik mimari, günümüzün ve geleceğin en zorlu e-ticaret hedeflerinden dahi hatasız, kesintisiz ve tespit edilemez bir veri akışını garanti edecektir.

Alıntılanan çalışmalar

1. Anti-Bot Bypass That Actually Works - Scrape.do, erişim tarihi Mayıs 17, 2026, <https://scrape.do/features/anti-bot-bypass/>
2. Web Scraping in 2025: Bypassing Modern Bot Detection | by Sohail x Codes | Medium, erişim tarihi Mayıs 17, 2026, https://medium.com/@sohail_saifii/web-scraping-in-2025-bypassing-modern-bot-detection-fcab286b117d
3. Browser agent bot detection is about to change, erişim tarihi Mayıs 17, 2026, <https://browser-use.com/posts/bot-detection>
4. The Hidden Fingerprints of Bot Protection: How Every Major Vendor ..., erişim tarihi Mayıs 17, 2026, <https://medium.com/@dimakynal/the-hidden-fingerprints-of-bot-protection-how-every-major-vendor-leaves-traces-in-your-browser-ae951e355606>
5. GitHub - microlinkhq/is-antibot: Detect anti-bot protection from 20+ providers — CloudFlare, Akamai, DataDome, PerimeterX, Kasada, Imperva, reCAPTCHA, hCaptcha, Turnstile, and more., erişim tarihi Mayıs 17, 2026, <https://github.com/microlinkhq/is-antibot>
6. Bypass Anti-Bot Protection - Cloudflare, Akamai, DataDome and More | Scrapfly, erişim tarihi Mayıs 17, 2026, <https://scrapfly.io/bypass>
7. Playwright Stealth: Bypass Bot Detection in Python & Node.js ..., erişim tarihi Mayıs 17, 2026, <https://scrapfly.io/blog/posts/playwright-stealth-bypass-bot-detection>
8. Scraping Ecommerce Sites Using LLMs - Part 1 · Neil's Blog, erişim tarihi Mayıs 17, 2026, <https://neilagrwal.com/post/creating-ecommerce-scrapers-part-1/>
9. Scraping E-Commerce Product Data - ScrapingBee, erişim tarihi Mayıs 17, 2026, <https://www.scrapingbee.com/blog/scraping-e-commerce-product-data/>
10. What are some decent rotating residential proxies for tough scraping targets? - Reddit, erişim tarihi Mayıs 17, 2026, https://www.reddit.com/r/scrapingtheweb/comments/1sgotuo/what_are_some_decent_rotating_residential_proxies/
11. TLS Fingerprinting: How It Works & How to Bypass It (2025), erişim tarihi Mayıs 17, 2026, <https://www.browserless.io/blog/tls-fingerprinting-explanation-detection-and-bypassing-it-in-playwright-and-puppeteer>
12. TLS Fingerprinting: Techniques and Bypassing Methods - Nstbrowser, erişim tarihi Mayıs 17, 2026, <https://www.nstbrowser.io/blog/tls-fingerprinting>
13. Overcoming TLS Fingerprinting in Web Scraping - Rayobyte, erişim tarihi Mayıs 17, 2026, <https://rayobyte.com/blog/tls-fingerprinting/>
14. How to avoid a bot detection and scrape a website using python? - Stack Overflow, erişim tarihi Mayıs 17, 2026,

- <https://stackoverflow.com/questions/68895582/how-to-avoid-a-bot-detection-and-scrape-a-website-using-python>
15. JA4 Fingerprinting Against AI Scrapers: A Practical Guide : r/netsec - Reddit, erişim tarihi Mayıs 17, 2026,
https://www.reddit.com/r/netsec/comments/1q71l7v/ja4_fingerprinting_against_ai_scrapers_a/
 16. Avoiding Bot Detection with Playwright Stealth - Bright Data, erişim tarihi Mayıs 17, 2026,
<https://brightdata.com/blog/how-tos/avoid-bot-detection-with-playwright-stealth>
 17. Avoid Bot Detection With Playwright Stealth: 9 Solutions for 2025, erişim tarihi Mayıs 17, 2026,
<https://www.scrapeless.com/en/blog/avoid-bot-detection-with-playwright-stealth>
 18. Baseline Performance Comparison of Nodriver, Zender, Selenium ..., erişim tarihi Mayıs 17, 2026,
<https://medium.com/@dimakynal/baseline-performance-comparison-of-nodriver-zender-selenium-and-playwright-against-anti-bot-2e593db4b243>
 19. NODRIVER vs Traditional Browser Automation Tools for Web Scraping - CapSolver, erişim tarihi Mayıs 17, 2026,
<https://www.capsolver.com/blog/All/nodriver-vs-traditional-browser-automation-tools>
 20. Alternative to undetected chromedriver? : r/webscraping, erişim tarihi Mayıs 17, 2026,
https://www.reddit.com/r/webscraping/comments/1lkr1cw/alternative_to_undetected_chromedriver/
 21. Best Undetected ChromeDriver Alternatives for 2026 - ZenRows, erişim tarihi Mayıs 17, 2026,
<https://www.zenrows.com/blog/undetected-chromedriver-alternatives>
 22. JA3/JA4 TLS Fingerprint - Detect Browser TLS/SSL Fingerprinting - Scrapfly, erişim tarihi Mayıs 17, 2026,
<https://scrapfly.io/web-scraping-tools/ja3-fingerprint?algo=ja4>
 23. Python HTTP Libraries Compared: Speed and Efficiency of `tls_client` and `curl_cffi` - Medium, erişim tarihi Mayıs 17, 2026,
<https://medium.com/@dimakynal/python-http-libraries-compared-speed-and-efficiency-of-tls-client-and-cffi-curl-7749d88e6502>
 24. Exploring Python Libraries: `tls_client` vs `curl_cffi` for Web Requests | by Dima Kynal | Medium, erişim tarihi Mayıs 17, 2026,
<https://medium.com/@dimakynal/exploring-python-libraries-tls-client-vs-curl-cffi-for-web-requests-129b7888e1f6>
 25. Stop using Python requests for web scraping: Use these modern modules instead - Zyte, erişim tarihi Mayıs 17, 2026,
<https://www.zyte.com/learn/stop-using-python-requests-for-web-scraping-use-these-modern-modules-instead/>
 26. Web Scraping With `curl_cffi` and Python in 2026 - Bright Data, erişim tarihi Mayıs

- 17, 2026, <https://brightdata.com/blog/web-data/web-scraping-with-curl-cffi>
27. I published a blazing-fast Python HTTP Client with TLS fingerprint : r/webscraping - Reddit, erişim tarihi Mayıs 17, 2026, https://www.reddit.com/r/webscraping/comments/1jdz7vo/i_published_a_blazingfast_python_http_client_with/
28. Building a Scalable Scraping Pipeline with Rotating Proxy Pools - DEV Community, erişim tarihi Mayıs 17, 2026, <https://dev.to/wisdomudo/building-a-scalable-scraping-pipeline-with-rotating-proxy-pools-1dc0>
29. How I've Built My Own Mobile Web Scraping Proxy | by Anthony Heath | Geek Culture, erişim tarihi Mayıs 17, 2026, <https://medium.com/geekculture/how-ive-built-my-own-mobile-web-scraping-proxy-728461597ce1>
30. What Are Rotating Proxies? Guide to IP Rotation for Web Scraping - WebScrapingAPI, erişim tarihi Mayıs 17, 2026, <https://www.webscrapingapi.com/web-scraping-rotating-proxies>
31. Bypassing the DOM: Extracting JSON-LD and Schema.org Metadata (2026) | Use Apify, erişim tarihi Mayıs 17, 2026, <https://use-apify.com/blog/schema-org-scraping-architecture>
32. LLM for web scraper: HTML structure analysis : r/LocalLLaMA - Reddit, erişim tarihi Mayıs 17, 2026, https://www.reddit.com/r/LocalLLaMA/comments/18nxu5c/llm_for_web_scraper_html_structure_analysis/
33. Structured Data for Ecommerce Sites | Google Search Central | Documentation, erişim tarihi Mayıs 17, 2026, <https://developers.google.com/search/docs/specialty/ecommerce/include-structured-data-relevant-to-ecommerce>
34. 7 Important Schema Markups for Ecommerce Websites in 2026 - ResultFirst, erişim tarihi Mayıs 17, 2026, <https://www.resultfirst.com/blog/ecommerce-seo/ecommerce-schema-markups/>
35. Product Schema: The Complete Guide to Product Structured Data and Merchant Listings, erişim tarihi Mayıs 17, 2026, <https://metaflow.life/blog/product-schema-guide>
36. Intro to Product Structured Data on Google | Google Search Central | Documentation, erişim tarihi Mayıs 17, 2026, <https://developers.google.com/search/docs/appearance/structured-data/product>
37. The Ultimate Guide to Structured Data for eCommerce Websites, erişim tarihi Mayıs 17, 2026, <https://thisisnovos.com/blog/the-ultimate-guide-to-structured-data-for-ecommerce-websites/>
38. Scrape Structured Data with Python and Extruct - Hackers and Slackers, erişim tarihi Mayıs 17, 2026, <https://hackersandslackers.com/scrape-metadata-json-ld/>
39. web scraping - How can I webscrape data from json-ld piece of code? - Stack Overflow, erişim tarihi Mayıs 17, 2026, <https://stackoverflow.com/questions/64820800/how-can-i-webscrape-data-fro>

[m-json-ld-piece-of-code](#)

40. Beyond BeautifulSoup: Benchmarking LLM-Powered Web Scraping for Everyday Users, erişim tarihi Mayıs 17, 2026, <https://arxiv.org/html/2601.06301v1>
41. Webscraper: Leverage Multimodal Large Language Models for Index-Content Web Scraping - arXiv, erişim tarihi Mayıs 17, 2026, <https://arxiv.org/html/2603.29161v1>
42. Top 5 Web Scraping Methods: Including Using LLMs - Comet, erişim tarihi Mayıs 17, 2026, <https://www.comet.com/site/blog/top-5-web-scraping-methods-including-using-llms/>